

Appel et retour de fonctions

Sommaire

Activations des fonctions

[Paramètres](#)

Blocs d'activation et pile

Un bloc d'activation est empilé à l'appel de la fonction et dépilé au retour

Les valeurs locales sont perdues

Séquence d'appel

Réserve et remplit un bloc d'activation

Séquence de retour

Restaure l'état de la machine avant l'appel pour reprendre l'exécution où elle en était

Code à trois adresses

```
tmp1 := $a
tmp2 := $4
tmp3 := b - tmp2
tmp4 := tmp3 * c
base[tmp1] := tmp4
```

x : adresse de la fonction appelée
y : nombre de paramètres

Forme générale : **x := y op z**

où **op** est un opérateur. Les trois adresses sont celles de **x**, **y** et **z**.

Instructions d'affectation

x := y op z **x := op y**
x := y **x := \$ k** (sans indirection)

Instructions d'appel

pour chaque paramètre **param x**
 appel **call x y**

Instructions de retour

return x **return**

Analyse sémantique des appels de fonctions

```
int partition(int y, int z) {  
    (...)  
  
    middle=partition(begin, end);  
}
```

Nombre et type des paramètres : comparer avec
la déclaration de la fonction
Calculer le type de la valeur renvoyée

Traduction des appels de fonctions en code intermédiaire

```
int partition(int y, int z) {  
    (...)  
  
    middle=partition(begin, end);  
}
```

Code évaluant les paramètres

Instructions **param**

Instruction **call**

Code intermédiaire

Code source

```
quicksort(0, n-1);
(...)
void quicksort(int f,int l){
int m;
(...)
m=partition(f, l);
quicksort(f, m-1);
quicksort(m+1, l);
return;
}
int partition(int f,int l){
(...)
return m;
}
```

```
param tmp_1
param tmp_2
call quicksort_3 2
(...)
quicksort_3:
(...)
param tmp_4
param tmp_5
call partition_6 2
(...)
param tmp_7
```

```
param tmp_8
call quicksort_3 2
(...)
param tmp_9
param tmp_10
call quicksort_3 2
return
partition_6:
(...)
return tmp_11
```

Fonctions et tables des symboles

```
int partition(int y, int z);  
(...)
```

```
middle=partition(begin, end);
```

Pour que les fonctions puissent être récursives, il faut pouvoir les appeler avant de les définir

En C : déclaration de prototype

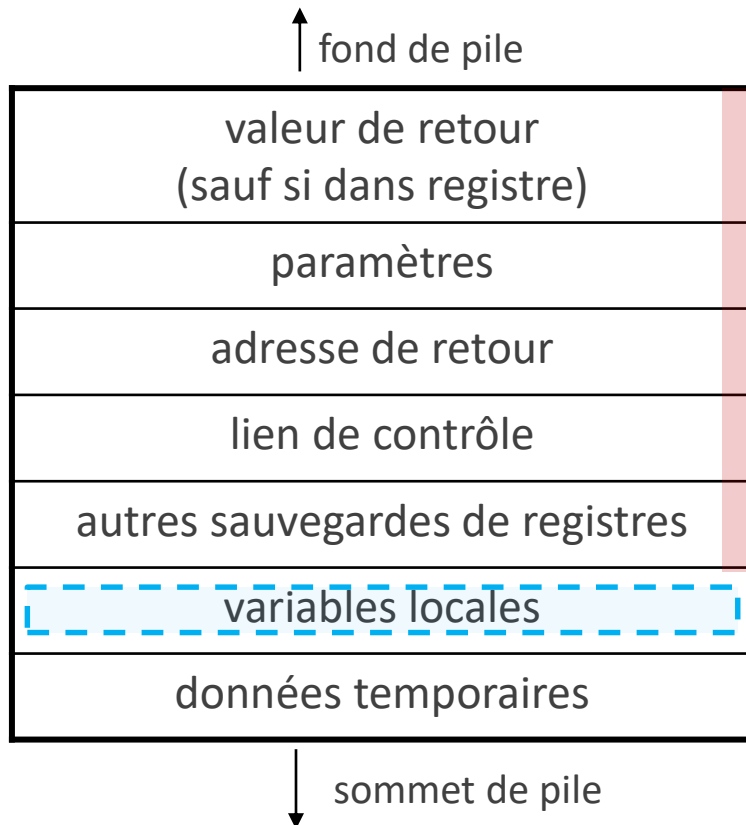
Dans le projet : pas de déclaration de prototype

2 parcours de l'arbre abstrait

- un pour les tables des symboles (y compris locales, à cause des paramètres)
- un pour traduire (y compris les appels)

Bloc d'activation (*stack frame*)

Zone de mémoire empilée à l'appel d'une fonction et dépilée au retour



Lien de contrôle : pointe sur la **base** du bloc d'activation appelant

Sauvegarde des **registres non volatils** au moment de l'appel

Données temporaires : pour le calcul des expressions

Pourquoi la valeur de retour au-dessous de tout le reste ?

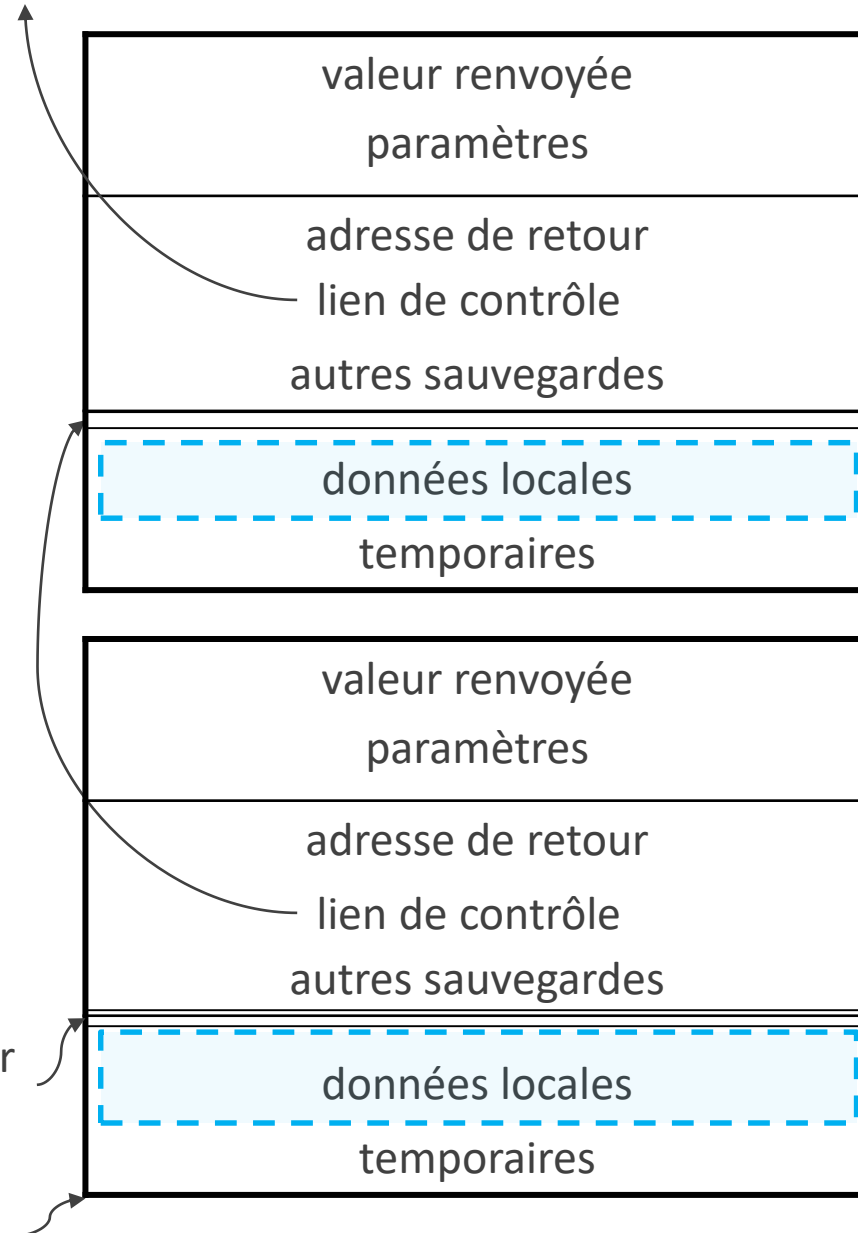
Pourquoi les paramètres au-dessous de l'adresse de retour ?

bloc d'activation de
l'appelant

bloc d'activation de
l'appelé

pointeur
de base

sommet
de pile



à la charge
de l'appelant

Exercice
 $y = f(g(x))$

Séquences d'appel et de retour

Opérations incluses dans
l'instruction `call` de
`nasm`

Séquence d'appel

L'appelant sauvegarde des **registres volatils**

Il aligne la pile

Il évalue les paramètres

Il **enregistre l'adresse de retour dans le bloc d'activation de l'appelé**

Il **saute à l'adresse de l'appelé**

L'appelé enregistre la valeur du pointeur de base dans le bloc d'activation de l'appelé

Il sauvegarde des registres **non volatils**

Il décrémente le pointeur de pile

Il initialise ses données locales

Opération incluse dans
l'instruction `ret` de `nasm`

Séquence de retour

L'appelé place la valeur à renvoyer

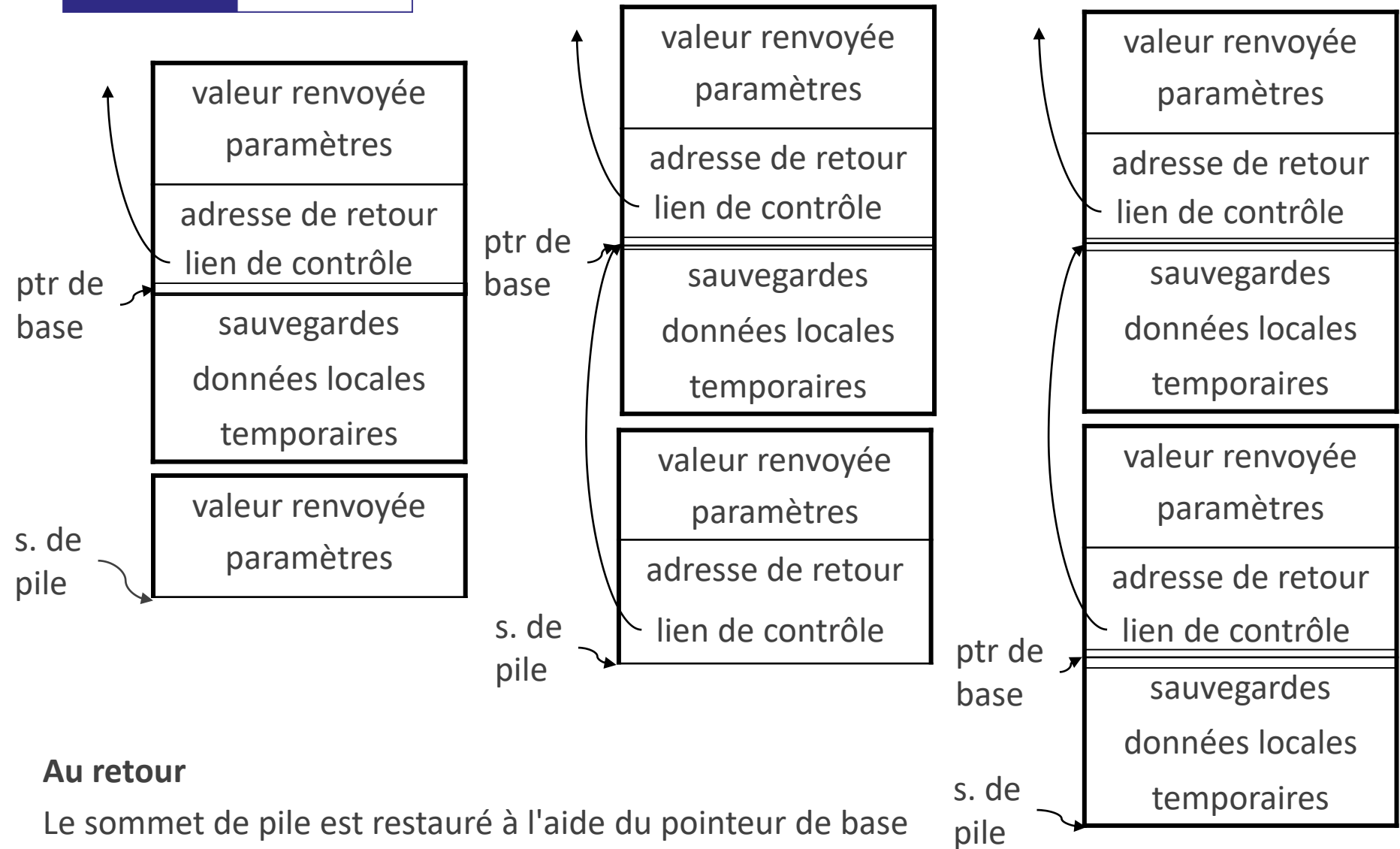
Il restaure les registres **non volatils** en terminant par le sommet de pile puis le pointeur de base

Il **saute à l'adresse de retour**

L'appelant restaure la pile

L'appelant utilise la valeur renvoyée

L'appelant restaure les **registres volatils**



Au retour

Le sommet de pile est restauré à l'aide du pointeur de base

Le pointeur de base est restauré à l'aide du lien de contrôle

Code intermédiaire

Code source

```
quicksort(0, n-1);
(...)
void quicksort(int f,int l){
int m;
(...)
m=partition(f, l);
quicksort(f, m-1);
quicksort(m+1, l);
return;
}
int partition(int f,int l){
(...)
return m;
}
```

```
param tmp_1
param tmp_2
call quicksort_3 2
(...)
quicksort_3:
(...)
param tmp_4
param tmp_5
call partition_6 2
(...)
param tmp_7
```

```
param tmp_8
call quicksort_3 2
(...)
param tmp_9
param tmp_10
call quicksort_3 2
return
partition_6:
(...)
return tmp_11
```

Traduction des appels de fonctions en code final

```
int partition(int y, int z) {  
    (...)  
  
    middle=partition(begin, end);  
}
```

Code sauvegardant des registres volatils

Code alignant la pile

Code évaluant les paramètres

Code plaçant les paramètres dans des registres

Instruction `call`

Code restaurant la pile

Code restaurant les registres volatils

Exercice : placer le code qui aligne la pile

Code intermédiaire

```
param tmp_4
param tmp_5
call partition_6 2
(...)
param tmp_7
param tmp_8
call quicksort_3 2
(...)
param tmp_9
param tmp_10
call quicksort_3 2
return
partition_6:
(...)
return tmp_11
```

Code final

```
pop rdi
pop rsi
call partition_6
push rax
(...)
pop rdi
pop rsi
call quicksort_3
(...)
pop rdi
pop rsi
call quicksort_3
```

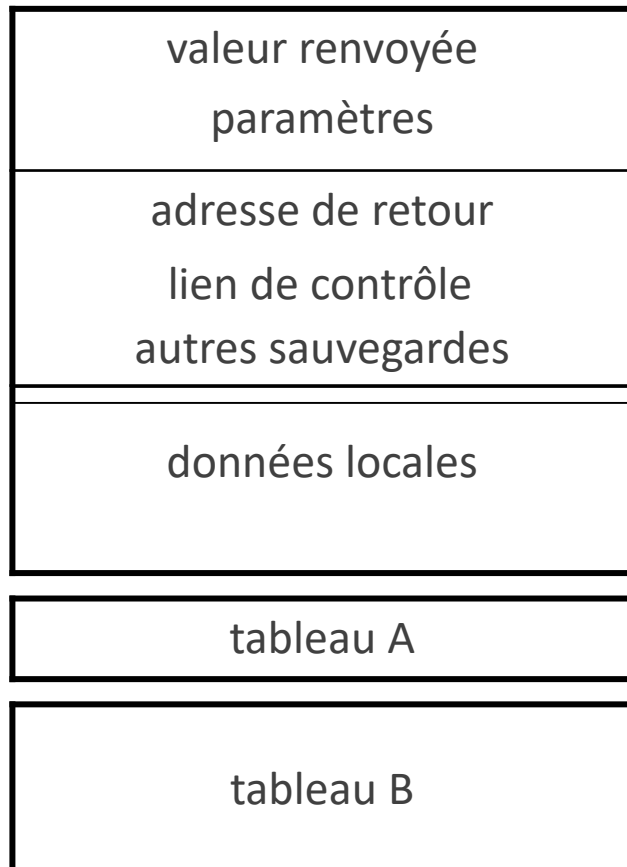
```
add rsp, 8
pop rbp
ret
partition_6:
push rbp
mov rbp, rsp
sub rsp, 16
(...)
pop rax
add rsp, 16
pop rbp
ret
```

Aligner la pile

```
mov r11, rsp
sub rsp, 8
and rsp, -16
mov qword [rsp], r11
call my_function2
pop rsp
```

← -16 étendu à 0ffff_ffff_ffff_fff0h

Allocation en pile et tableaux de longueur variable



Les tableaux du C sont représentés par des pointeurs

Tableau de longueur variable

Tableau dont la taille est inconnue à la compilation (Ada, C99)

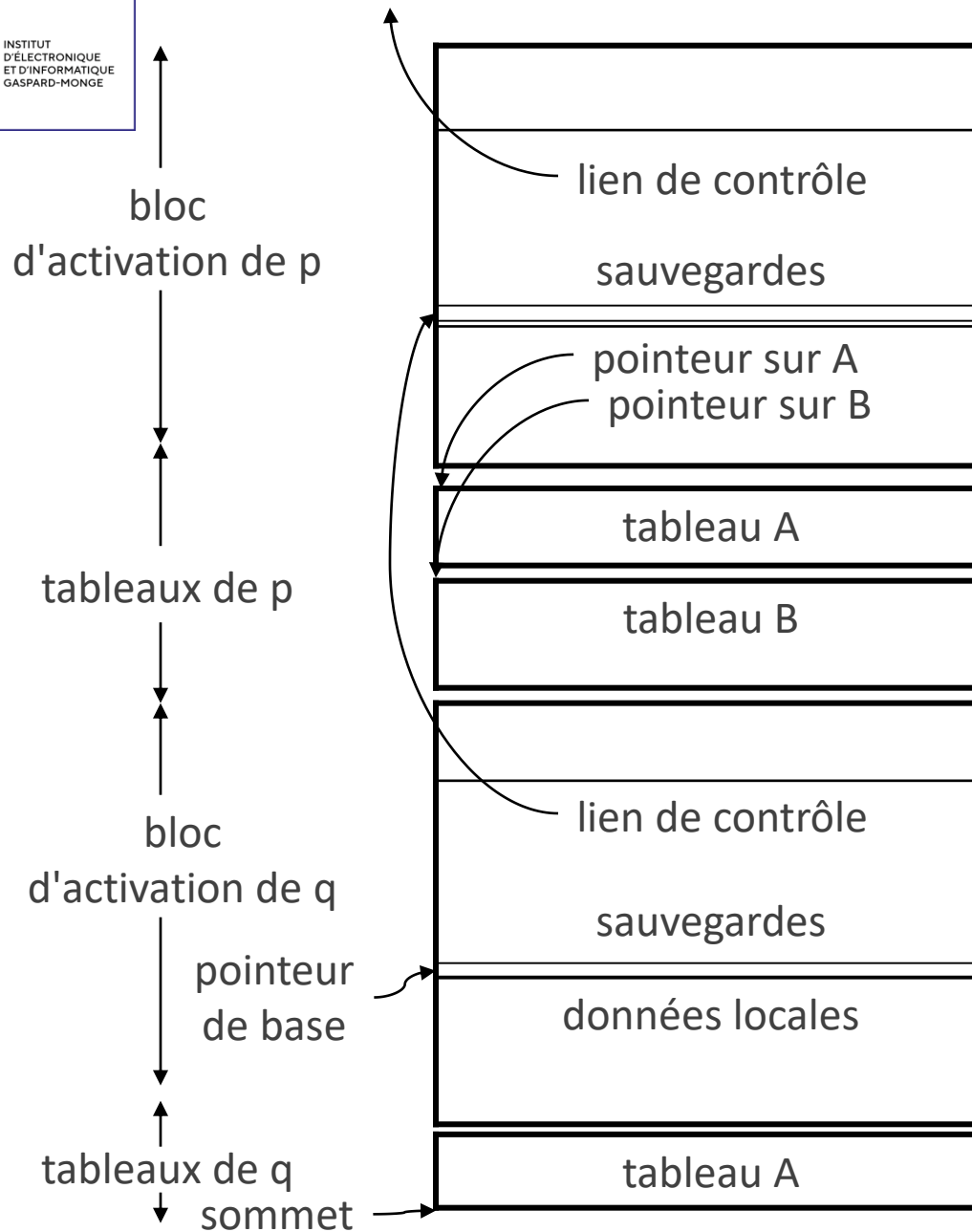
Les tableaux de longueur variable sont placés sur la pile au-dessus du bloc d'activation

Leur adresse relative n'est pas connue à la compilation

Le bloc d'activation contient des pointeurs vers ces tableaux

Les adresses relatives de ces pointeurs sont connues à la compilation

Le pointeur de
sommet de pile
tient compte
des données de
taille variable



Portée lexicale sans fonctions emboîtées

Exemple : le langage C

Les noms sont de deux sortes :

- locaux à une fonction ou à un bloc
- globaux, déclarés hors des fonctions

Les noms globaux sont alloués de façon statique

Les noms locaux sont en pile, accessibles à partir
du pointeur de base

Sauf les noms locaux statiques

Sommaire

[Activations des fonctions](#)

Paramètres

Paramètres

```
swap(int *x, int *y) {
    int temp ;
    temp = * x ;
    * x = * y ;
    * y = temp ;
}
main() {
    int a = 1, b = 2 ;
    swap(& a, &b) ;
}
```

Paramètres

Déclarations

Arguments

Expressions

Passage des paramètres

Dépend des langages

Par valeur : C, Java, C++, Ada

Par référence : Java, C++

Par copie/restauration : Ada, Fortran

Par nom : macros en C, Algol

Passage par valeur

```
swap(int i, int j) {  
    int temp ;  
    temp = a[i] ;  
    a[i] = a[j] ;  
    a[j] = temp ;  
}  
main() {  
    swap(1, 2) ;  
}
```

Les valeurs des paramètres effectifs sont passés à la procédure appelée

Exemples : C, Java, C++

Ne modifie pas les valeurs dans le bloc d'activation de l'appelant, sauf à travers des noms non locaux ou des pointeurs passés par valeur

Réalisation

Les paramètres formels sont traités comme des noms locaux

Les paramètres effectifs sont évalués par l'appelant

Passage par valeur

Ada

```
procedure Swap (i, j : Integer) is
    temp : Integer ;
begin
    temp := a(i) ;
    a(i) := a(j) ;
    a(j) := temp;
end Swap;
```

C

```
swap(int i, int j) {
    int temp ;
    temp = a[i] ;
    a[i] = a[j] ;
    a[j] = temp ;
}
main() {
    swap(a, 1, 2) ;
}
```

Passage par référence

```
void swap(int & x, int & y) {  
    int temp;  
    temp = x ;  
    x = y ;  
    y = temp;  
}
```

Les adresses des paramètres effectifs sont passées
à la procédure appelée

Java, C++

Exemple en C++


```

procedure My_program is
  a : integer ;
  procedure Test (x : in out integer) is
  begin
    x := 2 ;
    a := 3 ;
  end Test ;
begin
  a := 1 ;
  Test(a) ;
  Put(a, 3) ;
  New_Line ;
end My_program ;

```

Passage par copie et restauration

Les valeurs des paramètres effectifs sont passées à la procédure appelée

Au retour, les nouvelles valeurs des paramètres sont copiées à leur adresse

Résultats

par référence

3

par copie et restauration

2

Ada, Fortran

Passage par nom

```
#define intswap(a, b) { int t = a ; a = b ; b = t ; }
```

```
intswap(i, tab[i])
```

substitution :

```
{ int t = i ; i = tab[i] ; tab[i] = t ; }
```

équivalent à :

```
y=tab[i];
```

```
tab[tab[i]]=i;
```

```
i=y;
```

Passage par nom

Le corps de la fonction est substitué à l'appel

Les paramètres sont substitués littéralement

Exemples

Macros en C

Algol

Fonctions passées en paramètre

Différences entre langages

Un langage où on ne peut pas

- passer une fonction en paramètre d'une autre
- renvoyer une fonction comme valeur d'une autre :

APL

$m \leftarrow +/ (3 + \iota 4)$
m

Fonctions passées en paramètre

```
#include <stdio.h>
int m ;
int f(int n) { return m + n ; }
int g(int n) { return m * n ; }
int b(int (*h)(int)) { printf("%i\n", h(2)) ; }
int main(void) {
    m = 0 ;
    b(f) ; b(g) ; }
/* ou b(&f); b(&g); */
```

Possible en `nasm`

Comment passer une fonction en paramètre ou la renvoyer comme résultat

Copier l'adresse du code (l'étiquette)

Comment le moteur d'exécution appelle une fonction passée en paramètre

- il empile l'adresse de retour
- il saute à l'adresse du code (l'étiquette)

Fonctions passées en paramètre en C

```
#include <stdio.h>
int m ;
int f(int n) { return m + n ; }
int g(int n) { return m * n ; }
int b(int (*h)(int)) { printf("%i\n", h(2)) ; }
int main(void) {
    m = 0 ;
    b(f) ; b(g) ; }
/* ou b(&f); b(&g); */
```

Comment traduire en code intermédiaire le passage d'une fonction en paramètre ou le renvoi d'une fonction comme valeur de retour

On transmet l'adresse du code

Merci

CONTACT

ERIC LAPORTE

00 +33 (0)1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR